

Fundoo Notes App - Interview Preparation Guide

Comprehensive Q&A; on Architecture, Security, Spring Boot, JPA & Best Practices

Q1: Can you give an overview of your Fundoo Notes App project?

Answer: Fundoo Notes is a RESTful Backend Web Application built using **Spring Boot 3.2.5**, **Spring Data JPA**, **H2 In-Memory Database**, and **Spring Security 6**.

It provides User Management (Registration & Login) and Note Management (Create, Read, Update, Delete) with full user data isolation, ensuring users can only manage their own notes.

Q2: How did you implement authentication and security in this project?

Answer: We implemented Spring Security 6 with standard **HTTP Basic Authentication** (`.httpBasic()`) and a custom `UserDetailsService`.

- **Authentication:** Spring Security retrieves credentials (email & password) from request headers and verifies them against the database.
- **Password Storage:** For beginner/practice purposes, we configured `NoOpPasswordEncoder` so passwords are stored in plain text, making database debugging straightforward.

Q3: How do you handle Authorization and User Data Isolation?

Answer: Authorization is enforced in `NoteService` using JPA Entity Relationships:

- Each Note is mapped to a User via a `@ManyToOne` relationship.
- Before performing any CRUD operation (get, update, delete), `NoteService` checks if the authenticated user's email matches the email on the note.
- If User A tries to modify User B's note, an exception is thrown, returning **403 Forbidden**.

Q4: What is the purpose of `SecurityConfig.java` in your application?

Answer: `SecurityConfig` acts as the central security configuration class annotated with `@Configuration` and `@EnableWebSecurity`.

- **PasswordEncoder Bean:** Returns `NoOpPasswordEncoder.getInstance()` for plain-text password comparison.
- **UserDetailsService Bean:** Connects Spring Security to `UserRepository` to load users by email.
- **SecurityFilterChain Bean:** Disables CSRF, permits public endpoints (`/api/auth/**`, `/h2-console/**`, `/swagger-ui/**`), and secures `/api/notes/**` with HTTP Basic Auth.

Q5: What are the key Spring Boot & JPA annotations used in this project?

Answer:

- `@RestController` & `@RequestMapping`: Marks controllers and defines base URI paths.
- `@Service` & `@Repository`: Defines service layer logic and Spring Data JPA repositories.
- `@Entity`, `@Table`, `@Id`, `@GeneratedValue`: Maps Java models to database tables.
- `@ManyToOne` & `@JoinColumn(name = "user_id")`: Establishes foreign key relationship between Note and User.
- `@Valid` & `@RequestBody`: Validates request payload DTOs.

Q6: Why did you use an In-Memory Database (H2) for this application?

Answer: H2 is an lightweight, zero-configuration in-memory database.

- It allows instant startup and rapid testing without installing a standalone database server like MySQL or PostgreSQL.
- It comes with a built-in web console at /h2-console to inspect tables (users, notes) and SQL queries directly.

Q7: How would you upgrade this practice project to Production Level?

Answer: To make this application production-ready:

1. **Password Security:** Replace NoOpPasswordEncoder with BCryptPasswordEncoder.
2. **Token Authentication:** Replace HTTP Basic Auth with **JWT (JSON Web Tokens)** for stateless Bearer token authentication.
3. **Database Migration:** Switch H2 in-memory DB to **PostgreSQL / MySQL**.
4. **Global Exception Handling:** Implement @RestControllerAdvice for unified error responses.
5. **Pagination:** Use Spring Data Pageable for note lists.

Q8: How did you test your REST APIs?

Answer:

- **Postman & cURL:** Tested registration via POST /api/auth/register and executed note CRUD operations using Basic Auth (-u email:password).
- **Swagger UI:** Interactive documentation accessible at /swagger-ui.html.
- **Automated Testing:** Ran unit tests using JUnit 5 and Mockito (NoteServiceTest).